



南开大学
Nankai University

南 开 大 学

计 算 机 学 院

计算机网络实验报告

实验 2：设计可靠传输协议并编程实现

李昱

年级：2023 级

专业：计算机科学与技术

指导教师：吴英

2025 年 12 月 18 日

目录

一、 实验要求	1
二、 协议设计说明	1
(一) 协议报文格式设计	1
(二) 连接管理协议	1
三、 实现方法说明	2
(一) 数据结构与预处理	2
1. 发送缓冲区 (Client 端)	2
2. 接收缓冲区 (Server 端)	2
(二) 差错控制与校验	2
(三) 滑动窗口与流量控制实现	3
1. 发送窗口状态维护	3
2. 有效窗口计算与发送限制	4
3. 接收端窗口通告	4
(四) 拥塞控制 (Reno 算法实现)	4
1. ACK 处理与拥塞状态转移	4
(五) 超时重传 (RTO) 与 SACK 处理	5
1. 超时重传机制 (RTO)	5
2. SACK 选择性确认实现	6
四、 程序设计与模块划分	7
五、 程序实现	7
(一) 项目结构	7
(二) 关键功能代码示例	8
1. RENO 拥塞控制更新逻辑	8
2. 接收方 SACK 处理逻辑	8
六、 实验结果与分析	9
(一) 不同丢包率对传输性能的影响	9
(二) 不同接收窗口大小对传输性能的影响	9
(三) 实验结论	10
(四) 运行截图	10
(五) 仓库链接	11

一、 实验要求

利用数据报套接字在用户空间实现面向连接的可靠数据传输，功能包括：

- (1) 连接管理：包括建立连接、关闭连接和异常处理。
- (2) 差错检测：使用校验和进行差错检测。
- (3) 确认重传：支持流水线方式，采用选择确认。
- (4) 流量控制：发送窗口和接收窗口使用相同的固定大小窗口。
- (5) 拥塞控制：实现 RENO 算法。

二、 协议设计说明

(一) 协议报文格式设计

为了在 UDP 之上实现可靠传输，本实验自定义了应用层协议头部。协议采用紧凑的二进制打包方式，通过 `#pragma pack(1)` 指令强制 1 字节对齐，确保在网络传输时内存布局一致。

核心数据结构 Message 由头部 Header 和数据负载 data 组成。其中 Header 的结构如下表所示：

字段	类型	描述
seq_num	uint32_t	序列号 ：对于数据包，表示该分段在文件中的字节偏移量（代码中为 <code>bytes_read + 1</code> ）；对于握手/挥手包，使用逻辑序号。
ack_num	uint32_t	确认号 ：采用累积确认机制，表示期望收到的下一个字节序列号。
length	uint16_t	数据长度 ：仅指 data 部分的有效字节数，最大不超过 MSS (1024)。
checksum	uint16_t	校验和 ：覆盖头部和数据部分的 16 位反码和，用于检测传输错误。
flags	uint8_t	标志位 ：按位定义的控制标志。 • SYN (0x01)：请求建立连接。 • ACK (0x02)：确认收到有效包。 • FIN (0x04)：请求释放连接。 • DATA (0x08)：标识包含文件数据。
window_size	uint8_t	接收窗口通告 ：接收端将剩余缓冲区大小（单位：包数量）填入此字段，用于流量控制。
sack_start	uint32_t	SACK 起始序号 ：当接收端检测到乱序包时，在此填入乱序块的起始序列号。
sack_end	uint32_t	SACK 结束序号 ：在此填入乱序块的结束序列号（左闭右开区间）。

(二) 连接管理协议

1. 建立连接（三次握手）：

- Client 发送 SYN 包, Seq=0。
- Server 收到后, 回复 SYN | ACK 包, Ack=1。
- Client 收到回复后, 发送 ACK 包, Seq=1, Ack=1。连接建立, 后续文件数据 Seq 从 1 开始。

2. 断开连接 (四次挥手):

- Client 发送 FIN | ACK 包, 进入等待状态。
- Server 收到 FIN 后, 回复 ACK, 随后发送自己的 FIN | ACK。
- Client 收到 Server 的 FIN 后, 发送最后的 ACK 并关闭。
- Server 收到最后的 ACK 后关闭连接。

三、 实现方法说明

(一) 数据结构与预处理

1. 发送缓冲区 (Client 端)

客户端定义了 PacketInfo 结构体来管理发送缓冲区:

Listing 1: 发送缓冲区元数据

```
1 struct PacketInfo {  
2     Message msg;  
3     bool is_acked;      // 是否已被SACK或累积ACK确认  
4     std::chrono::steady_clock::time_point send_time; // 上次发送时间 (用于计算  
5         RTO)  
6     bool sent_once;     // 是否已经发送过至少一次  
};
```

所有待发送的数据包被预先加载到 `vector<PacketInfo> buffer` 中, 便于通过下标索引进行随机访问和重传。

2. 接收缓冲区 (Server 端)

服务器端利用 C++ STL 的 `std::map<uint32_t, Message>` 实现乱序接收缓冲区。

- **自动排序**: map 根据 Key (序列号) 自动排序, 简化了对乱序块的检测。
- **去重**: 如果收到重复的序列号, map 的特性或逻辑检查可防止重复插入。

(二) 差错控制与校验

代码中实现了 Internet Checksum 算法:

Listing 2: 计算并设置校验和 (由发送方调用)

```
1 void set_checksum() {  
2     head.checksum = 0;  
3     uint32_t sum = 0;  
4     uint16_t* p = (uint16_t*)this;
```

```

5 // 计算 Header + 数据部分的 16位字和
6 int size_bytes = sizeof(Header) + head.length;
7 for (int i = 0; i < size_bytes / 2; i++) {
8     sum += *p++;
9     while (sum >> 16) sum = (sum & 0xffff) + (sum >> 16);
10 }
11 // 处理奇数字节
12 if (size_bytes % 2 != 0) {
13     uint16_t tmp = (*(uint8_t*)p);
14     sum += tmp;
15     while (sum >> 16) sum = (sum & 0xffff) + (sum >> 16);
16 }
17 head.checksum = ~(sum & 0xffff); // 取反
18 }

```

Listing 3: 验证校验和 (由接收方调用)

```

1 bool check_checksum() {
2     uint32_t sum = 0;
3     uint16_t* p = (uint16_t*)this;
4     int size_bytes = sizeof(Header) + head.length;
5     for (int i = 0; i < size_bytes / 2; i++) {
6         sum += *p++;
7         while (sum >> 16) sum = (sum & 0xffff) + (sum >> 16);
8     }
9     if (size_bytes % 2 != 0) {
10         uint16_t tmp = (*(uint8_t*)p);
11         sum += tmp;
12         while (sum >> 16) sum = (sum & 0xffff) + (sum >> 16);
13     }
14     return (sum & 0xffff) == 0xffff; // 结果应为全1
15 }

```

1. 将头部和数据看作 16 位整数序列。
2. 计算所有 16 位整数的二进制反码和。
3. 如果数据字节数为奇数，最后填充一个字节 0 处理。
4. 发送端调用 `set_checksum()` 填充字段；接收端调用 `check_checksum()`，若计算结果为 0xFFFF 则校验通过，否则丢弃。

(三) 滑动窗口与流量控制实现

基于滑动窗口的流量控制机制，可以确保发送速率既不超过网络的拥塞限制，也不超过接收端的处理能力。

1. 发送窗口状态维护

客户端维护两个关键指针来管理发送缓冲区：`base` 指向最早发出但未被确认的数据包（窗口左沿），`next_seq` 指向下一个待发送的数据包。

Listing 4: 发送窗口指针定义

```

1 // 传输状态变量
2 int base = 0;          // 发送窗口基准索引 (最早未确认的包)
3 int next_seq = 0;      // 下一个待发送的包索引

```

2. 有效窗口计算与发送限制

发送方在每一轮循环中, 取拥塞窗口 (cwnd) 和接收方通告窗口 (rwnd) 的最小值作为当前有效窗口大小 (effective_window)。发送循环严格受限于此窗口大小, 只有当 next_seq 在窗口范围内时才发送新数据。

Listing 5: 流量控制与发送循环限制

```

1 // 发送逻辑: 有效窗口 = min(拥塞窗口, 接收方通告窗口)
2 int effective_window = min((int)cwnd, (int)rwnd);
3
4 // 仅在窗口范围内发送数据
5 while (next_seq < buffer.size() && next_seq < base + effective_window) {
6     if (!buffer[next_seq].is_acked) {
7         if (!buffer[next_seq].sent_once) {
8             send_udp(buffer[next_seq].msg);
9             // ... 记录发送时间 ...
10        }
11    }
12    next_seq++;
13 }

```

3. 接收端窗口通告

服务器端在构建 ACK 包时, 会根据当前乱序缓冲区 (out_of_order_buffer) 的占用情况, 计算剩余可用的接收窗口大小, 并填入协议头的 window_size 字段。这实现了实时的流量反馈, 防止发送方溢出接收缓冲区。

Listing 6: 接收端计算剩余窗口

```

1 // 流量控制: 通告剩余接收窗口大小
2 // RCV_WND 为总缓冲区大小 (20), 减去当前已占用的乱序包数量
3 ack_msg.head.window_size = (uint8_t)(RCV_WND - out_of_order_buffer.size());
4
5 send_msg(ack_msg);

```

(四) 拥塞控制 (Reno 算法实现)

本实验在客户端实现了完整的 TCP Reno 拥塞控制状态机, 通过维护拥塞窗口 (cwnd)、慢启动阈值 (ssthresh) 和重复 ACK 计数器 (dup_ack_count) 来动态调整发送速率。

1. ACK 处理与拥塞状态转移

客户端在收到 ACK 后, 根据确认号 ack_num 判断是新确认还是重复确认, 从而进入不同的拥塞控制阶段。

1. **收到新 ACK**：当 `ack_val > base_seq_num` 时，表示有新数据被确认。系统根据当前 `cwnd` 与 `ssthresh` 的关系，选择执行“慢启动”或“拥塞避免”算法。

Listing 7: 慢启动与拥塞避免实现

```
1 if (ack_val > base_seq_num) {
2     // ...更新 Base 指针...
3     dup_ack_count = 0; // 重置重复ACK计数
4
5     if (cwnd < ssthresh) {
6         // 每收到一个 ACK，窗口加 1（即一个 RTT 内翻倍）
7         cwnd += 1.0;
8     } else {
9         // 每个 RTT 增加 1 MSS（每收到一个 ACK 增加 1/cwnd）
10        cwnd += 1.0 / cwnd;
11    }
12 }
```

2. **收到重复 ACK**：当收到重复确认时，累加计数器。若计数达到 3 次，则判定发生了轻微拥塞（数据包丢失但网络未断），立即触发“快速重传”和“快速恢复”，而不是等待超时。

Listing 8: 快速重传与快速恢复实现

```
1 else {
2     // Duplicate ACK
3     dup_ack_count++;
4
5     if (dup_ack_count == 3) {
6         // [快速重传] 立即重传丢失的 Base 包
7         send_udp(buffer[base].msg);
8         buffer[base].send_time = std::chrono::steady_clock::now();
9
10        // [快速恢复]
11        // 1. 阈值减半
12        ssthresh = max(cwnd / 2, 2.0);
13        // 2. 拥塞窗口设置为 阈值 + 3（考虑已收到的3个冗余包）
14        cwnd = ssthresh + 3;
15    }
16 }
```

（五） 超时重传 (RTO) 与 SACK 处理

除了基于 ACK 的反馈，实验还实现了基于时间的兜底重传机制以及基于 SACK 的选择性重传优化。

1. 超时重传机制 (RTO)

客户端在主循环中实时检测窗口基准包 (base) 的发送时间。一旦超过设定的 `INITIAL_RTO` (500ms)，系统判定网络严重拥塞，执行最严格的窗口重置策略。

Listing 9: RTO 超时处理逻辑

```

1 // 计算距离上次发送的时间差
2 auto time_since_send = std::chrono::duration_cast<std::chrono::milliseconds>(
    now - buffer[base].send_time).count();
3
4 if (buffer[base].sent_once && time_since_send > INITIAL_RTO) {
5     // [超时惩罚]
6     ssthresh = max(cwnd / 2, 2.0); // 阈值减半
7     cwnd = 1.0; // 拥塞窗口重置为 1 (进入慢启动)
8     dup_ack_count = 0;
9
10    // 重传 Base 包
11    send_udp(buffer[base].msg);
12    buffer[base].send_time = std::chrono::steady_clock::now();
13
14    // Go-Back-N 策略回退, 但结合 SACK 只会重发未确认的包
15    next_seq = base + 1;
16 }

```

2. SACK 选择性确认实现

为了避免重传已到达的数据包, 协议引入了 SACK 机制。

1. **Server 端生成 SACK**: 当接收到乱序包时, 服务器将其存入 map 缓冲区, 并在 ACK 包中通告已接收的连续块范围。

Listing 10: Server 端 SACK 构造

```

1 // 如果收到乱序包, 存入缓冲区
2 else if (seq > expected_seq) {
3     out_of_order_buffer[seq] = recv_msg;
4 }
5
6 // 填充 SACK 字段: 通告缓冲区中第一个连续块的 [Start, End)
7 if (!out_of_order_buffer.empty()) {
8     ack_msg.head.sack_start = out_of_order_buffer.begin()->first;
9     ack_msg.head.sack_end = ack_msg.head.sack_start + out_of_order_buffer.
        begin()->second.head.length;
10 }

```

2. **Client 端处理 SACK**: 客户端收到带有 SACK 信息的 ACK 后, 遍历发送缓冲区, 将处于 SACK 范围内的包标记为已确认。在后续的重传或发送循环中, 已标记的包将被跳过。

Listing 11: Client 端 SACK 处理

```

1 // SACK 处理 (标记乱序收到的包)
2 if (ack_pkt.head.sack_start > 0 && ack_pkt.head.sack_end > 0) {
3     for (int i = base; i < next_seq; i++) {
4         uint32_t pkt_seq = buffer[i].msg.head.seq_num;
5         // 如果包序列号在 SACK 范围内, 标记为已确认

```



```

6         if (pkt_seq >= ack_pkt.head.sack_start && pkt_seq < ack_pkt.head.
            sack_end) {
7             buffer[i].is_acked = true;
8         }
9     }
10 }

```

四、 程序设计与模块划分

程序分为客户端（发送方）和服务端（接收方）两个独立模块：

• Client 模块 (发送方):

- 读取本地文件并切分为 MSS 大小的分段。
- 维护发送滑动窗口，根据 cwnd 和 rwnd 限制发送速率。
- 实现 RENO 状态机，根据 ACK 反馈调整窗口大小。
- 处理超时重传逻辑。

• Server 模块 (接收方):

- 维护接收缓冲区 `std::map<uint32_t, Message>` 处理乱序到达的数据包。
- 实现累积确认逻辑，更新 `expected_seq`。
- 通过 `LOSS_RATE` 参数模拟真实网络环境中的随机丢包。
- 将接收到的连续数据写入目标文件。

五、 程序实现

(一) 项目结构

```

lab2:.
|   client.cpp           # 发送方实现 (Reno, SACK, Sliding Window)
|   client.exe           # 编译后的发送方可执行文件
|   server.cpp           # 接收方实现 (SACK 生成, 乱序处理, 模拟丢包)
|   server.exe           # 编译后的接收方可执行文件
|   test_data.bin        # 待传输测试文件
|   实验报告.pdf
|

```

\---运行截图

握手与挥手.png

快速重传触发.png

丢包率对比.jpg

(二) 关键功能代码示例

1. RENO 拥塞控制更新逻辑

Listing 12: 拥塞窗口更新

```

1  if (ack_val > base_seq_num) {
2      // 收到新数据确认
3      while (base < buffer.size() && buffer[base].msg.head.seq_num < ack_val) {
4          buffer[base].is_acked = true;
5          base++;
6      }
7      dup_ack_count = 0;
8      if (cwnd < ssthresh) {
9          cwnd += 1.0; // 慢启动
10     } else {
11         cwnd += 1.0 / cwnd; // 拥塞避免
12     }
13 } else {
14     // 收到重复ACK
15     dup_ack_count++;
16     if (dup_ack_count == 3) {
17         // 快速重传与快恢复
18         send_udp(buffer[base].msg);
19         ssthresh = max(cwnd / 2, 2.0);
20         cwnd = ssthresh + 3;
21     }
22 }

```

2. 接收方 SACK 处理逻辑

Listing 13: 接收方乱序缓冲与 SACK 生成

```

1  if (seq == expected_seq) {
2      out_file.write(recv_msg.data, len);
3      expected_seq += len;
4      // 检查缓冲区后续连续包...
5  } else if (seq > expected_seq) {
6      out_of_order_buffer[seq] = recv_msg; // 存入有序 Map
7  }
8  // 生成 SACK 信息通告发送方
9  if (!out_of_order_buffer.empty()) {
10     ack_msg.head.sack_start = out_of_order_buffer.begin()->first;
11     ack_msg.head.sack_end = ack_msg.head.sack_start + out_of_order_buffer.
        begin()->second.head.length;
12 }

```

六、实验结果与分析

本次实验使用大小为 10,485,760 Bytes (约 10 MB) 的二进制文件进行传输测试。实验环境为 localhost 本地回环测试。

(一) 不同丢包率对传输性能的影响

测试条件：接收窗口大小固定为 $RCV_WND = 20$, $MSS = 1024$ Bytes。

通过修改服务器端的 $LOSS_RATE$ 参数，模拟不同程度的网络丢包环境，记录传输时间与平均吞吐率，数据如下表所示：

表 1: 不同丢包率下的传输性能对比

丢包率 (%)	传输时间 (s)	平均吞吐率 (MB/s)	平均吞吐率 (Kbps)
0	0.19	51.43	431,387
1	5.78	1.73	14,515
5	80.59	0.12	1,041
10	328.44	0.03	255

结果分析：

- **0% 丢包率 (基准)：**在理想环境下，传输仅受限于本地 I/O 和 CPU 处理速度，实现了约 51.4 MB/s 的极高吞吐率。此时拥塞窗口 $cwnd$ 呈指数增长（慢启动）直至线性增长（拥塞避免），且无重传发生。
- **1% 丢包率：**性能出现断崖式下跌（下降约 30 倍）。这是因为即使只有 1% 的丢包，也会导致发送方频繁收到重复 ACK。虽然 Reno 算法的“快速重传”机制避免了部分超时，但频繁的丢包仍导致 $cwnd$ 频繁减半，无法维持在大窗口状态。
- **5% - 10% 丢包率：**性能恶化，传输时间从几秒延长至几分钟。在高丢包率下，一个窗口内往往丢失多个数据包，导致接收方无法发送足够的重复 ACK 触发快速重传，从而只能等待 RTO 超时 (Timeout)。RTO 触发后， $cwnd$ 重置为 1，进入慢启动，导致吞吐率急剧下降。

(二) 不同接收窗口大小对传输性能的影响

测试条件：丢包率固定为 $LOSS_RATE = 2\%$, $MSS = 1024$ Bytes。

保持 2% 的随机丢包率，调整接收端通告窗口 RCV_WND 的大小，观察其对传输的影响：

表 2: 2% 丢包率下不同接收窗口的性能对比

接收窗口 (包)	传输时间 (s)	平均吞吐率 (MB/s)	平均吞吐率 (Kbps)
5	11.87	0.84	7,065
20	18.40	0.54	4,559
50	13.36	0.75	6,279

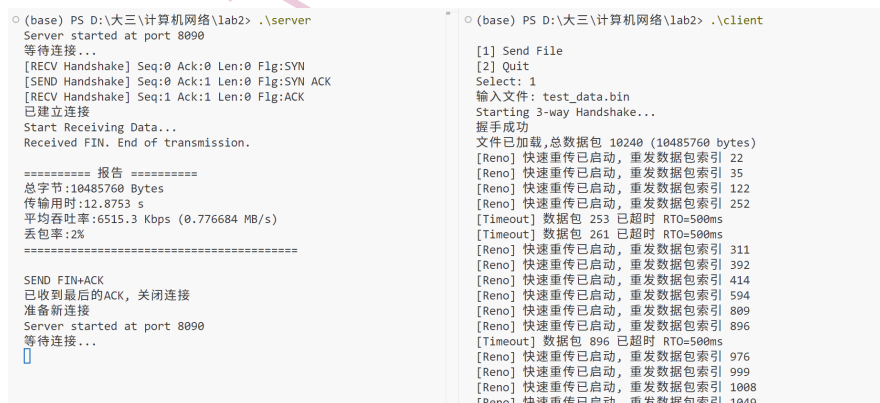
结果分析：

- **小窗口优势 (Win=5):** 实验数据显示, 在 2% 丢包环境下, 较小的窗口 (5) 取得了最高的吞吐率 (0.84 MB/s)。
- **原因:**
 1. **减少超时概率:** 窗口较小时, 在飞 (In-flight) 的数据包数量少。一旦发生丢包, 需要重传的数据量也少, 也不容易发生“一个窗口内大量丢包”的情况, 从而减少了 RTO 超时重传 (导致 $cwnd=1$) 的发生频率。
 2. **大窗口的代价:** 当窗口为 20 或 50 时, 发送方倾向于并发发送大量数据。在 2% 的丢包率下, 这意味着一个 RTT 内可能会丢失多个包。虽然 SACK 能够优化重传, 但如果丢失过多, Reno 算法容易退化为超时重传, 导致“停-等”现象严重。
- **Win=20 与 Win=50 的对比:** 窗口从 20 增加到 50 时, 性能有所回升 (0.54 $\rightarrow$ 0.75 MB/s)。可能是因为更大的窗口在未发生丢包的时间片内能更充分地利用带宽, 或者更大的 SACK 块能涵盖更多的乱序数据, 一定程度上抵消了重传的开销。

(三) 实验结论

1. **可靠性验证:** 无论丢包率如何变化, 文件最终均能完整传输且 MD5 校验一致, 证明了基于校验和、序列号、ACK 和重传机制的可靠性设计是成功的。
2. **Reno 算法特性:** 实验清晰地展示了 Reno 算法对丢包的敏感性。随着丢包率上升, 机制从“快速重传/恢复”逐渐退化为“超时重传”, 验证了 TCP 拥塞控制在有损网络下的行为特征。
3. **流量控制权衡:** 在存在随机丢包的网络中, 盲目增大接收窗口并不一定能提高吞吐率, 反而可能因频繁触发拥塞控制的惩罚机制而降低性能。

(四) 运行截图



```
(base) PS D:\大三\计算机网络\lab2> .\server
Server started at port 8090
等待连接...
[RECV Handshake] Seq:0 Ack:0 Len:0 Flg:SYN
[SEND Handshake] Seq:0 Ack:1 Len:0 Flg:SYN ACK
[RECV Handshake] Seq:1 Ack:1 Len:0 Flg:ACK
已建立连接
Start Receiving Data...
Received FIN. End of transmission.

===== 报告 =====
总字节:10485760 Bytes
传输用时:12.8753 s
平均吞吐率:6515.3 Kbps (0.776684 MB/s)
丢包率:2%
=====

SEND FIN+ACK
已收到最后的ACK, 关闭连接
准备新连接
Server started at port 8090
等待连接...

(base) PS D:\大三\计算机网络\lab2> .\client
[1] Send File
[2] Quit
Select: 1
输入文件: test_data.bin
Starting 3-way Handshake...
握手成功
文件已加载, 总数据 10240 (10485760 bytes)
[Reno] 快速重传已启动, 重发数据包索引 22
[Reno] 快速重传已启动, 重发数据包索引 35
[Reno] 快速重传已启动, 重发数据包索引 122
[Reno] 快速重传已启动, 重发数据包索引 252
[Timeout] 数据包 253 已超时 RTT=500ms
[Timeout] 数据包 261 已超时 RTT=500ms
[Reno] 快速重传已启动, 重发数据包索引 311
[Reno] 快速重传已启动, 重发数据包索引 392
[Reno] 快速重传已启动, 重发数据包索引 414
[Reno] 快速重传已启动, 重发数据包索引 594
[Reno] 快速重传已启动, 重发数据包索引 809
[Reno] 快速重传已启动, 重发数据包索引 896
[Timeout] 数据包 896 已超时 RTT=500ms
[Reno] 快速重传已启动, 重发数据包索引 976
[Reno] 快速重传已启动, 重发数据包索引 999
[Reno] 快速重传已启动, 重发数据包索引 1008
[Reno] 快速重传已启动, 重发数据包索引 1049
```

图 1: 客户端与服务端建立连接过程

```
[2] Quit
Select: 1
输入文件: test_data.bin
Starting 3-way Handshake...
握手成功
文件已加载,总数据包 10240 (10485760 bytes)
[Reno] 快速重传已启动, 重发数据包索引 13
[Reno] 快速重传已启动, 重发数据包索引 148
[Reno] 快速重传已启动, 重发数据包索引 172
[Reno] 快速重传已启动, 重发数据包索引 406
[Timeout] 数据包 408 已超时 RTO=500ms
[Reno] 快速重传已启动, 重发数据包索引 569
[Reno] 快速重传已启动, 重发数据包索引 655
[Reno] 快速重传已启动, 重发数据包索引 1052
[Reno] 快速重传已启动, 重发数据包索引 1107
[Reno] 快速重传已启动, 重发数据包索引 1348
[Reno] 快速重传已启动, 重发数据包索引 1373
[Reno] 快速重传已启动, 重发数据包索引 1562
[Reno] 快速重传已启动, 重发数据包索引 1734
```

图 2: 快速重传与超时重传触发

```
(base) PS D:\大三\计算机网络\lab2> .\server
Server started at port 8090
等待连接...
[RECV Handshake] Seq:0 Ack:0 Len:0 Flg:SYN
[SEND Handshake] Seq:0 Ack:1 Len:0 Flg:SYN ACK
[RECV Handshake] Seq:1 Ack:1 Len:0 Flg:ACK
已建立连接
Start Receiving Data...
Received FIN. End of transmission.

===== 报告 =====
总字节:10485760 Bytes
传输用时:0.216414 s
平均吞吐率:387619 Kbps (46.2078 MB/s)
丢包率:0%
=====

(base) PS D:\大三\计算机网络\lab2> .\server
Server started at port 8090
等待连接...
[RECV Handshake] Seq:0 Ack:0 Len:0 Flg:SYN
[SEND Handshake] Seq:0 Ack:1 Len:0 Flg:SYN ACK
[RECV Handshake] Seq:1 Ack:1 Len:0 Flg:ACK
已建立连接
Start Receiving Data...
Received FIN. End of transmission.

===== 报告 =====
总字节:10485760 Bytes
传输用时:74.5957 s
平均吞吐率:1124.54 Kbps (0.134056 MB/s)
丢包率:5%
```

图 3: 0% 丢包率 (上) 与 5% 丢包率 (下) 的性能对比

(五) 仓库链接

[Github 仓库](#)